

마이크로 프로세서 스탑워치 / 타이머만들기 REPORT

2025. 06. 17

담당교수	전자공학부 이진
학과	전자공학부
학번	2022142046
이름	최다빈

1.프로젝트 목표

본 프로젝트는 ATmega128 MCU 기반 보드에서 Microchip Studio를 사용하여 스톱워치 및 타이머 시스템을 구현하는 것을 목표로 합니다. 사용자는 PC의 시리얼 터미널(Tera Term)을 통해 명령을 전송하여 '스톱워치 모드'와 '타이머 모드'를 선택하고 제어할 수 있습니다. 시스템의 핵심 기능은 7-세그먼트 표시 장치, 외부 푸시 버튼, 그리고 인터럽트 기반의 실시간 처리입니다.

주요 기능

모드 전환: 시리얼 명령(M0, M1)을 통한 모드 변경.

스톱워치: PD0 버튼으로 시작/정지, PD1 버튼으로 리셋 기능 수행.

타이머: 시리얼 명령(T+시간, TP, TR)으로 시간 설정, 일시정지, 재개 기능 수행.

추가 기능: PORTB 버튼 입력 시 특정 시간(200ms ~ 800ms) 동안 지연을 발생시키는 기능.

표시: 모든 시간은 7-세그먼트에 0.01초 단위로 표시됩니다.

2. 시스템 구성

하드웨어

MCU: ATmega128

컴파일러: avr-gcc (Microchip Studio 환경)

디스플레이: 4자리 7-세그먼트 FND (데이터: PORTA, 자릿수 제어: PORTC).

입력 장치

푸시 버튼 2개 (PORTD의 PD0, PD1) - 스톱워치 제어용.

푸시 버튼 (PORTB) - 지연 기능 테스트용.

통신: PC와 USART1 시리얼 통신 (9600 bps).

소프트웨어

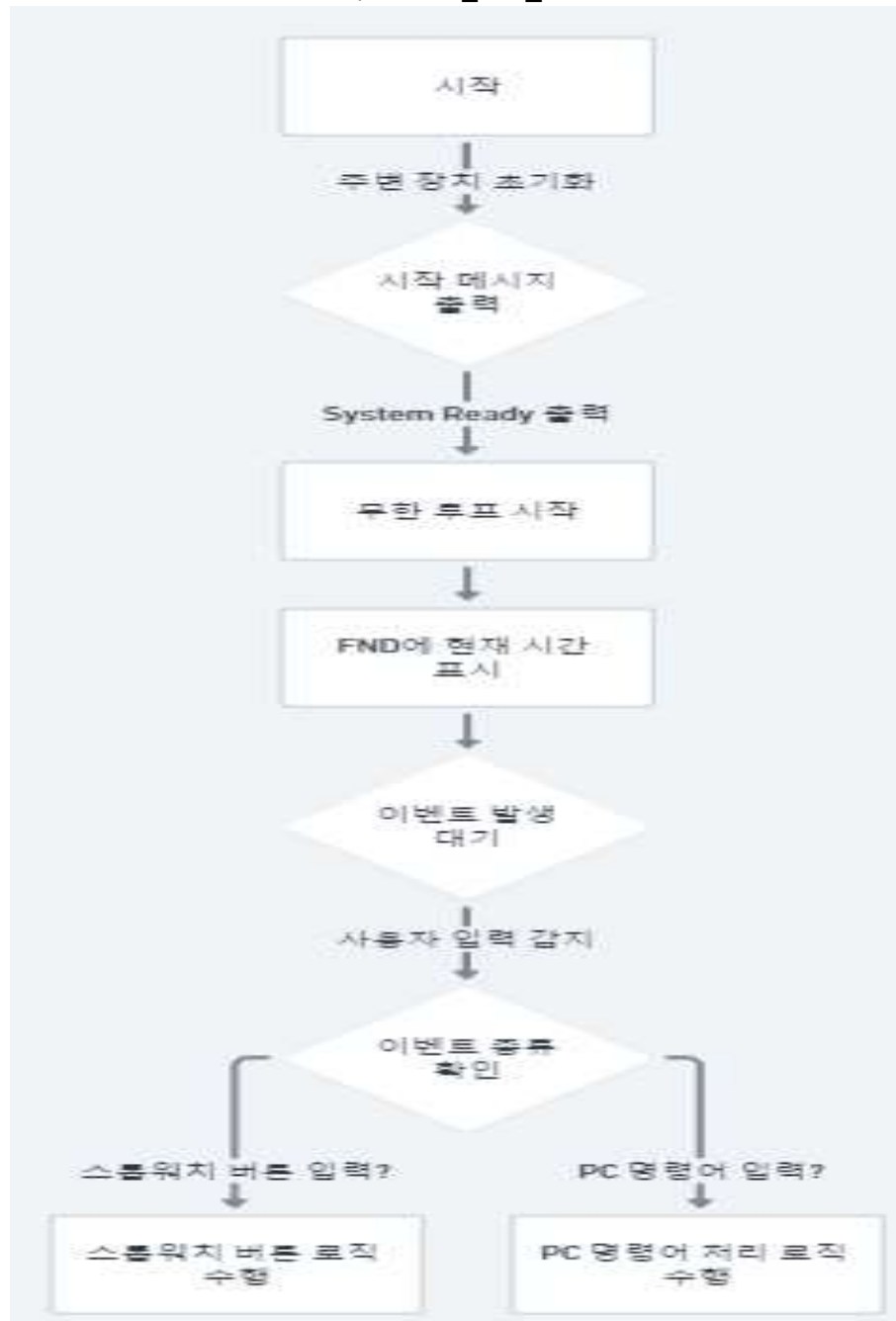
개발 환경: Microchip Studio

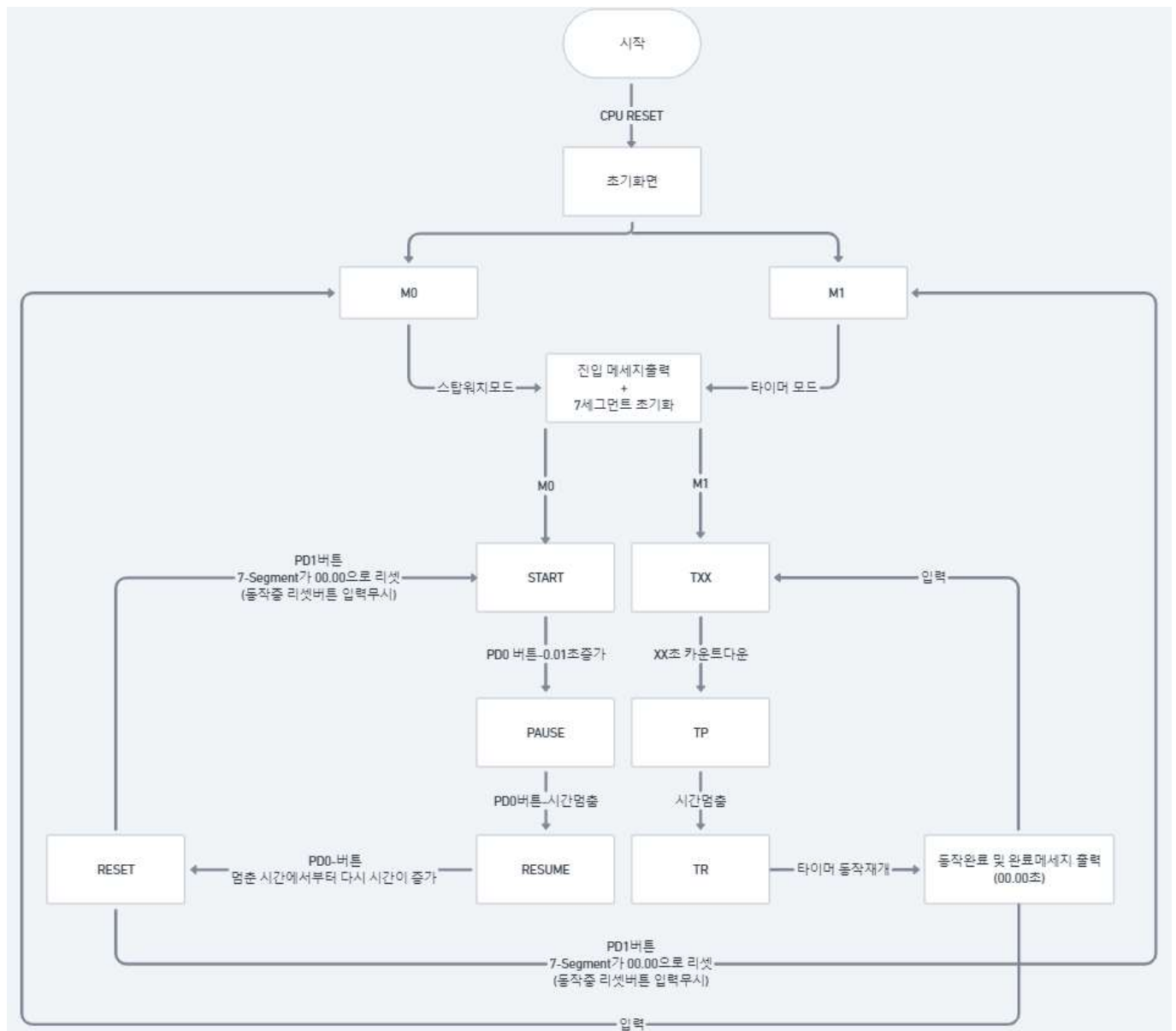
터미널 프로그램: Tera Term

3. 동작 설명

입력 종류	명령/버튼	기능 설명
시리얼 명령	M0	스톱워치 모드로 전환하고, 7-세그먼트를 00.00으로 초기화합니다 1D .
	M1	타이머 모드로 전환하고, 7-세그먼트를 00.00으로 초기화합니다 1D .
	T+시간 (예: T30)	타이머 시간을 초 단위(1~99초)로 설정하고 카운트다운을 시작합니다 1D 3D .
	TP	타이머 동작을 일시정지합니다 1D .
	TR	일시정지된 타이머를 재개합니다 1D .
버튼 입력	PD0 (스톱워치 모드)	스톱워치를 **시작(Start)**하거나 **일시정지(Pause)**합니다 1D 3D .
	PD1 (스톱워치 모드)	일시정지 상태에서 스톱워치를 **리셋(Reset)**하여 00.00으로 만듭니다. (동작 중에는 무시됨) 1D 3D .
	PORTB 버튼	버튼에 따라 200ms, 400ms, 600ms, 800ms의 지연을 발생시키고 터미널에 실행 메시지를 출력합니다 3D .

4. 프로그램 흐름도





5. 주요 소스 코드 설명

```
// --- 메인 함수 ---
int main(void) {
    init_ports();
    init_usart1();
    init_timer1_10ms();
    usart1_tx_string("\r\n=== System Ready [FINAL-DEBUGGED] ===\r\n");
    reset_system_state();
    sei();
    while (1) {
        handle_buttons();
        display_fnd(g_time_counter);
    }
}
```

메인 함수 (main())

시스템 초기화

- init_ports(): 포트 초기화 (FND, 버튼 설정)
- init_usart1(): USART1 통신 초기화
- init_timer1_10ms(): 타이머1 초기화 (10ms 주기 설정)
- System Ready 메시지를 PC 터미널에 출력
- reset_system_state(): 내부 변수 초기화 및 기본 상태 설정
- sei(): 전역 인터럽트 활성화

메인 루프 (while(1))

- 버튼 입력 처리 (handle_buttons()) → 스톱워치 시작/정지/리셋 기능 수행
- 현재 g_time_counter값을 FND에 표시
- 무한 반복하여 지속적인 동작 유지

```
// --- 인터럽트 서비스 루틴 ---
ISR(TIMER1_COMPA_vect) {
    if (!g_is_running) return;
    if (g_current_mode == MODE_STOPWATCH) {
        if (g_time_counter < 9999) g_time_counter++;
        else g_is_running = 0;
    } else if (g_current_mode == MODE_TIMER) {
        if (g_time_counter > 0) g_time_counter--;
        if (g_time_counter == 0) {
            g_is_running = 0;
            usart1_tx_string("\r\n>> Timer Complete! <<\r\n");
        }
    }
}
```

타이머 인터럽트 (ISR(TIMER1_COMPA_vect))

10ms마다 자동 실행

g_is_running 변수가 0이면 즉시 종료

현재 모드에 따라 다르게 동작

스톱워치 모드

g_time_counter가 9999 미만이면 1 증가

9999에 도달하면 타이머 중지

타이머 모드 (MODE_TIMER)

g_time_counter가 0보다 크면 1 감소

g_time_counter가 0이 되면 타이머 중지, PC 터미널에 "Timer Complete!" 메시지 출력

```

ISR(USART1_RX_vect) {
    char received_char = UDR1;

    // [수정된 부분] 수신된 문자를 터미널에 즉시 되돌려 보냄 (에코 기능)
    usart1_tx_char(received_char);

    if (g_rx_index < RX_BUFFER_SIZE - 1) {
        if (received_char == '\r' || received_char == '\n') {
            g_rx_buffer[g_rx_index] = '\0';
            if (g_rx_index > 0) process_command();
            g_rx_index = 0;
        } else {
            g_rx_buffer[g_rx_index++] = received_char;
        }
    } else {
        g_rx_index = 0;
    }
}

```

PC 명령어 수신 인터럽트 (ISR(USART1_RX_vect))

PC로부터 데이터가 수신되면 실행

UDR1에서 문자 1개를 읽음

수신된 문자를 다시 PC로 출력

명령어를 버퍼에 저장

Enter ('Wr' 또는 'Wn') 입력 시

문자열 종료

명령어 분석 및 실행

버퍼 인덱스 초기화

```

void process_command(void) {
    char cmd[RX_BUFFER_SIZE];
    strcpy(cmd, (const char*)g_rx_buffer);

    // '\r' 문자가 버퍼에 남아있을 수 있으므로 제거
    cmd[strcspn(cmd, "\r")] = 0;

    if (strcmp(cmd, "M0") == 0) {
        g_current_mode = MODE_STOPWATCH;
        reset_system_state();
    } else if (strcmp(cmd, "M1") == 0) {
        g_current_mode = MODE_TIMER;
        reset_system_state();
    } else if (g_current_mode == MODE_TIMER) {
        if (cmd[0] == 'T' && cmd[1] >= '0' && cmd[1] <= '9') {
            int seconds = atoi(&cmd[1]);
            if (seconds > 0 && seconds <= 99) {
                g_time_counter = (uint16_t)seconds * 100;
                g_is_running = 1;
                char msg[40];
                sprintf(msg, "\r\n[Timer Start] Set to %d seconds.\r\n", seconds);
                uart1_tx_string(msg);
            } else {
                uart1_tx_string("\r\n[Error] Invalid time. Use 1-99 seconds.\r\n");
            }
        } else if (strcmp(cmd, "TP") == 0) {
            g_is_running = 0;
            uart1_tx_string("\r\n[Timer Paused]\r\n");
        } else if (strcmp(cmd, "TR") == 0) {
            if (g_time_counter > 0) {
                g_is_running = 1;
                uart1_tx_string("\r\n[Timer Resumed]\r\n");
            }
        }
    }
}

```

명령어 처리 함수 (process_command())

시리얼 명령어 분석 및 실행

"M0" → 스톱워치 모드로 변경 및 초기화

"M1" → 타이머 모드로 변경 및 초기화

타이머 모드일 때 추가 기능 지원

"T+시간" → 타이머 시작 (1~99초 설정 가능)

g_time_counter = (uint16_t)seconds * 100

"Timer Start" 메시지를 PC에 출력

"TP" → 타이머 일시정지 (g_is_running = 0)

"TR" → 타이머 재개 (g_is_running = 1 단, g_time_counter > 0일 때만 가능)

6. 느낀점/개선점

배운 점

인터럽트 기반의 멀티태스킹: 이번 프로젝트의 핵심은 인터럽트였습니다.

10ms마다 정확하게 시간을 측정하는 타이머 인터럽트(TIMER1_COMPA_vect)와 사용자의 비동기적 입력을 처리하는 UART 수신 인터럽트(USART1_RX_vect)를 동시에 활용했습니다.

이를 통해 메인 루프가 다른 작업을 수행하는 동안에도 백그라운드에서 시간 계산과 시리얼 통신이 끊임 없이 이루어지는, 효율적인 시스템을 구축하는 방법을 체득했습니다.

Blocking 방식(예: `_delay_ms`)에만 의존하던 초기 단계에서 벗어나, 논블로킹(Non-blocking) 방식의 프로그래밍이 왜 중요한지 실감할 수 있었습니다.

하드웨어 제어 및 응용: ATmega128의 다양한 주변 장치를 직접 제어하는 경험은 이론 지식을 실제 코드로 연결하는 중요한 계기가 되었습니다.

타이머/카운터 설정: 14.7456MHz 클럭과 64분주비를 이용해 정확히 10ms 주기를 만들어내는 OCR1A 값(2304)을 계산하고 CTC 모드를 설정하는 과정에서 데이터시트를 분석하고 응용하는 능력을 길렀습니다.

GPIO 활용: FND(7-세그먼트)의 동적 디스플레이(Dynamic Display)를 위해 데이터 포트(PORTA)와 자릿수 제어 포트(PORTC)를 빠르게 전환하는 방법을 익혔고, 버튼 입력을 위해 내부 풀업 저항을 설정(PORTD, PORTB)하는 등 GPIO의 세부 기능을 활용했습니다.

USART 통신: PC와 안정적으로 데이터를 주고받기 위해 보레이트(Baud Rate), 프레임 포맷 등을 설정하고, 수신 버퍼를 직접 관리하며 문자열 기반의 명령어 처리 시스템을 구현했습니다.

상태 관리 기법: '스톱워치/타이머 모드'와 '실행/정지 상태'를 `g_current_mode`, `g_is_running` 같은 volatile 전역 변수로 관리하며 간단한 형태의 상태 머신(State Machine)을 구현했습니다.

시스템의 현재 상태에 따라 코드의 흐름이 어떻게 바뀌어야 하는지 설계하는 능력이 향상되었습니다.

느낀 점

디버깅의 중요성: 임베디드 시스템은 PC 환경처럼 디버깅이 쉽지 않았습니다. 원하는 대로 동작하지 않을 때, UART 통신을 통해 특정 변수의 값이나 함수 호출 여부를 터미널에 출력하는 방식이 시스템 내부를 들여다볼 수 있는 유일한 창이었습니다.

이 과정을 통해 논리적이고 체계적인 디버깅 접근법의 중요성을 절실히 느꼈습니다.

통합의 어려움과 성취감: 처음에는 각 기능(FND, 버튼, UART)이 독립적으로 잘 동작했지만, 이들을 하나의 시스템으로 통합하는 과정에서 예상치 못한 문제들이 발생했습니다.

특히 인터럽트와 메인 루프의 작업이 서로에게 영향을 미치지 않도록 코드를 조율하는 것이 까다로웠습니다. 모든 기능이 요구사항에 맞게 유기적으로 동작하는 최종 결과물을 확인했을 때 큰 성취감을 느낄 수 있었습니다.

부족한 점

Blocking 방식의 한계: 버튼의 채터링(Chattering)을 방지하기 위해 사용한 `_delay_ms(50)` 함수는 가장 간단한 해결책이었지만, 딜레이가 발생하는 동안 MCU가 다른 어떤 작업도 하지 못하는 Blocking 방식이라는 한계가 있었습니다.

만약 더 정밀한 실시간 처리가 필요했다면 이 방식은 문제가 될 수 있었을 것입니다.

코드의 모듈성 부족: 모든 기능이 하나의 `final.c` 파일 안에 구현되어 있어 코드가 길어지고 복잡해졌습니다. FND 제어, 명령어 처리, 하드웨어 초기화 등 기능별로 파일을 분리하여 모듈화했다면 코드의 가독성과 재사용성이 훨씬 높아졌을 것입니다.

견고하지 않은 입력 처리: 시리얼 명령어 처리가 T+숫자 외의 잘못된 입력에 대해 충분히 대응하지 못합니다. 사용자의 다양한 오류를 예상하고 예외 처리를 추가하여 시스템 안정성을 높이는 부분이 부족했습니다.

향후 개선 가능성

Non-Blocking 디바운싱 적용: 타이머 인터럽트를 활용한 상태 머신 기반의 디바운싱(Debouncing) 기법을 도입하여, `_delay_ms()` 호출 없이도 안정적으로 버튼 입력을 처리하도록 개선할 수 있습니다. 이는 시스템의 전반적인 반응성을 높일 것입니다.

코드 리팩토링 및 모듈화: `fnd.c`, `uart.c`, `cli.c`(명령어 처리기) 등으로 파일을 분리하고, 각 모듈이 헤더 파일(.h)을 통해 인터페이스를 제공하는 방식으로 구조를 개선하고 싶습니다. 이를 통해 보다 전문적이고 유지보수가 용이한 프로젝트로 발전시킬 수 있습니다.

기능 확장

알람 기능: 타이머가 완료되었을 때 단순히 메시지를 출력하는 것을 넘어, 피에조 부저(Piezo Buzzer)를 이용해 알람 소리를 내는 기능을 추가할 수 있습니다.

EEPROM 연동: 전원이 꺼져도 마지막 모드나 타이머 설정 값을 기억하도록 ATmega128의 내장 EEPROM을 활용하는 기능을 추가할 수 있습니다.

번호	테스트 내용	동작 확인	영상 시점 (분:초)
1	M0 입력 시 <u>스탑워치</u> 모드 진입	O	00:04
2	<u>스탑워치</u> Start / Pause / Reset 동작	O	00:09
3	M1 입력 시 타이머 모드 진입	O	00:51
4	타이머 기본 동작	O	01:02
5	타이머 TP / TR 명령 동작	O	01:51
6	<u>스탑워치</u> 모드 동작 중 M1 입력으로 타이머 모드 진입	O	02:20
7	타이머 모드 동작 중 M0 입력으로 <u>스탑워치</u> 모드 진입	O	02:46

동영상파일첨부

<https://youtu.be/BXPriBF8fVo>